

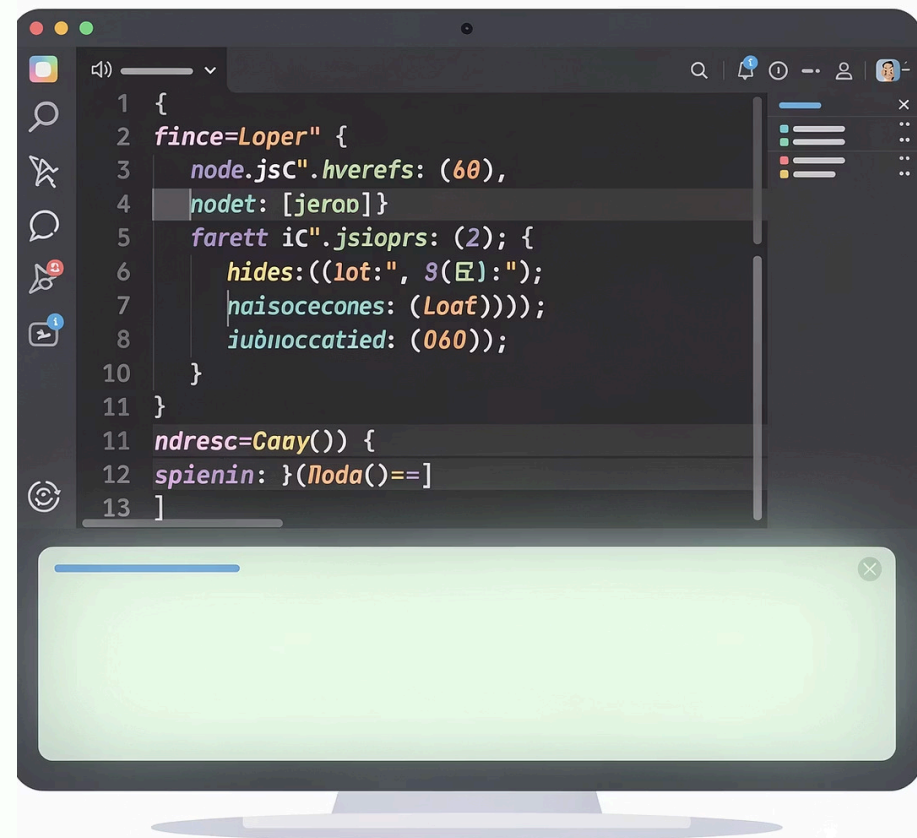
ОСНОВИ JavaScript + Telegraf

У цьому модулі ти напишеш справжнього Telegram-бота на Node.js – з нуля до робочого коду.

NODE.JS

TELEGRAF

JAVASCRIPT



Мінімальний робочий бот

```
const { Telegraf } = require('telegraf');
const bot = new Telegraf('ТВИЙ_ТОКЕН');

bot.start((ctx) => {
  ctx.reply('Бот запущений!');
});

bot.launch();
```

require('telegraf')

Підключає
бібліотеку Telegraf

new Telegraf(...)

Створює екземпляр
бота з токеном

bot.start(...)

Обробляє команду
/start

bot.launch()

Запускає бота – без
цього нічого не
працює!

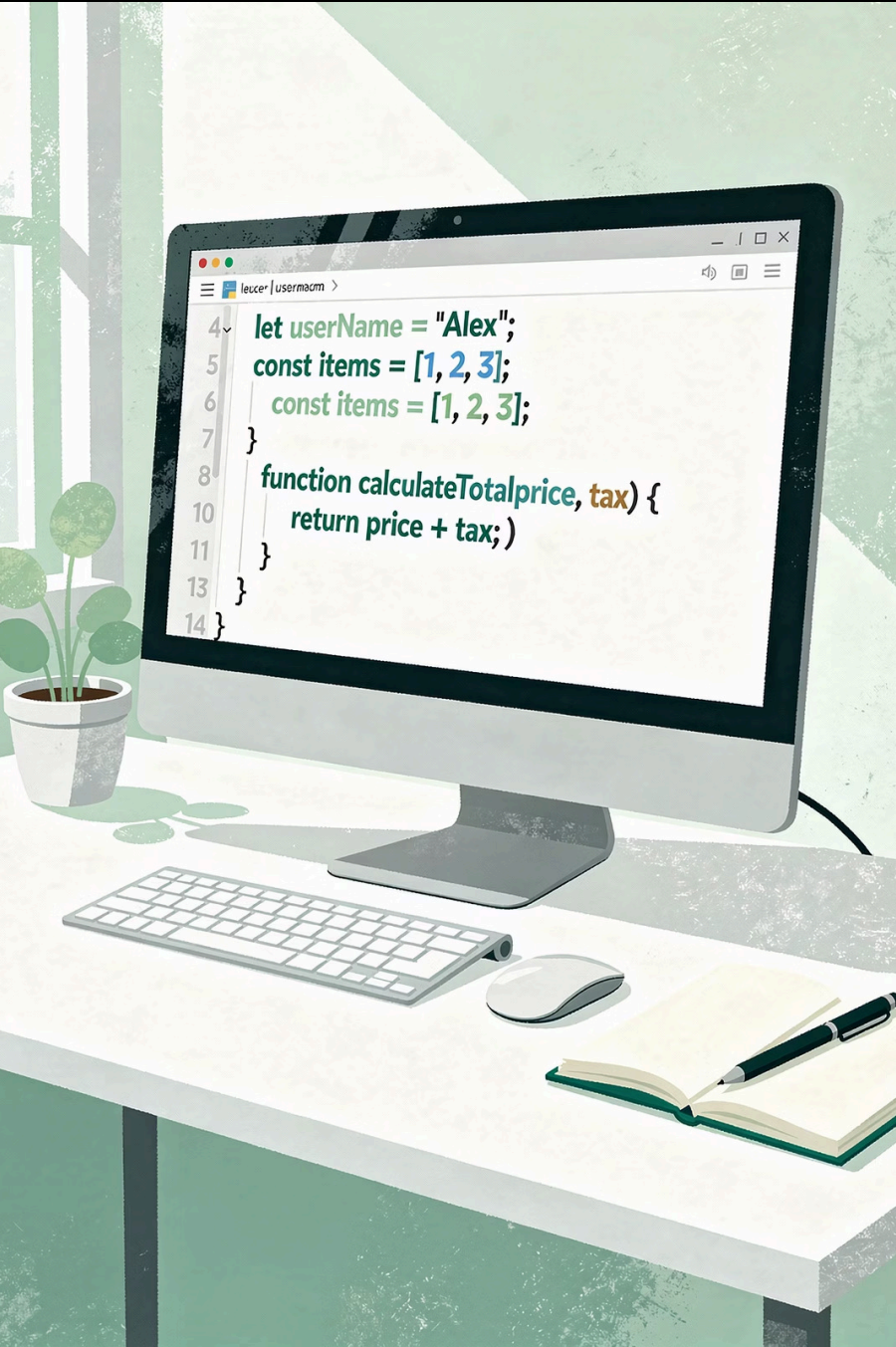
Дебаг через `console.log`

```
bot.start((ctx) => {  
  console.log(  
    "Отримано /start від:",  
    ctx.from.first_name  
  );  
  ctx.reply("Привіт!");  
});
```

Навіщо це потрібно?

- Бачиш, що відбувається всередині бота
- Перевіряєш, чи дішов запит
- Знаходиш помилки до того, як вони зламають усе

📄 **Термінал — твій найкращий друг при розробці.**



Змінні у боті

```
bot.start((ctx) => {  
  const userName =  
    ctx.from.first_name;  
  const message = "Привіт,  
    " + userName + "!";  
  ctx.reply(message);  
});
```

const

Значення не змінюється
після присвоєння

let

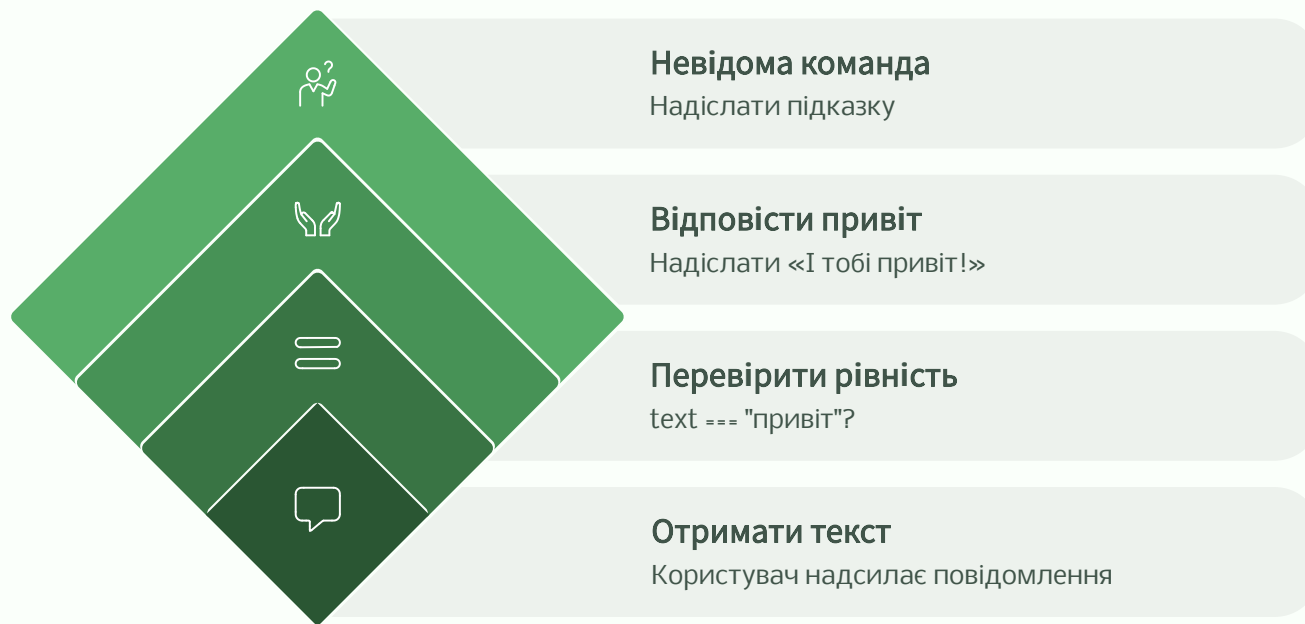
Значення можна
змінювати пізніше

`ctx.from.first_name` – ім'я
користувача прямо з Telegram

Умови if/else

```
bot.on("text", (ctx) => {  
  const text = ctx.message.text;  
  
  if (text === "привіт") {  
    ctx.reply("І тобі привіт!");  
  } else {  
    ctx.reply("Я поки знаю тільки  
'привіт'");  
  }  
});
```

Як бот приймає рішення



Функції для структури коду

```
function getWelcomeMessage(name) {  
  return "Ласкаво просимо, " + name;  
}  
  
bot.start((ctx) => {  
  const name = ctx.from.first_name;  
  ctx.reply(getWelcomeMessage(name));  
});
```

Навіщо виносити логіку у функції?

- Код стає читабельнішим
- Можна повторно використовувати в різних місцях
- Легше знайти і виправити помилку



Одна функція — одна задача. Це основа чистого коду.

Масиви відповідей

```
const jokes = [
```

"Чому програмісти не виходять на вулицю? Бо там немає Wi-Fi."

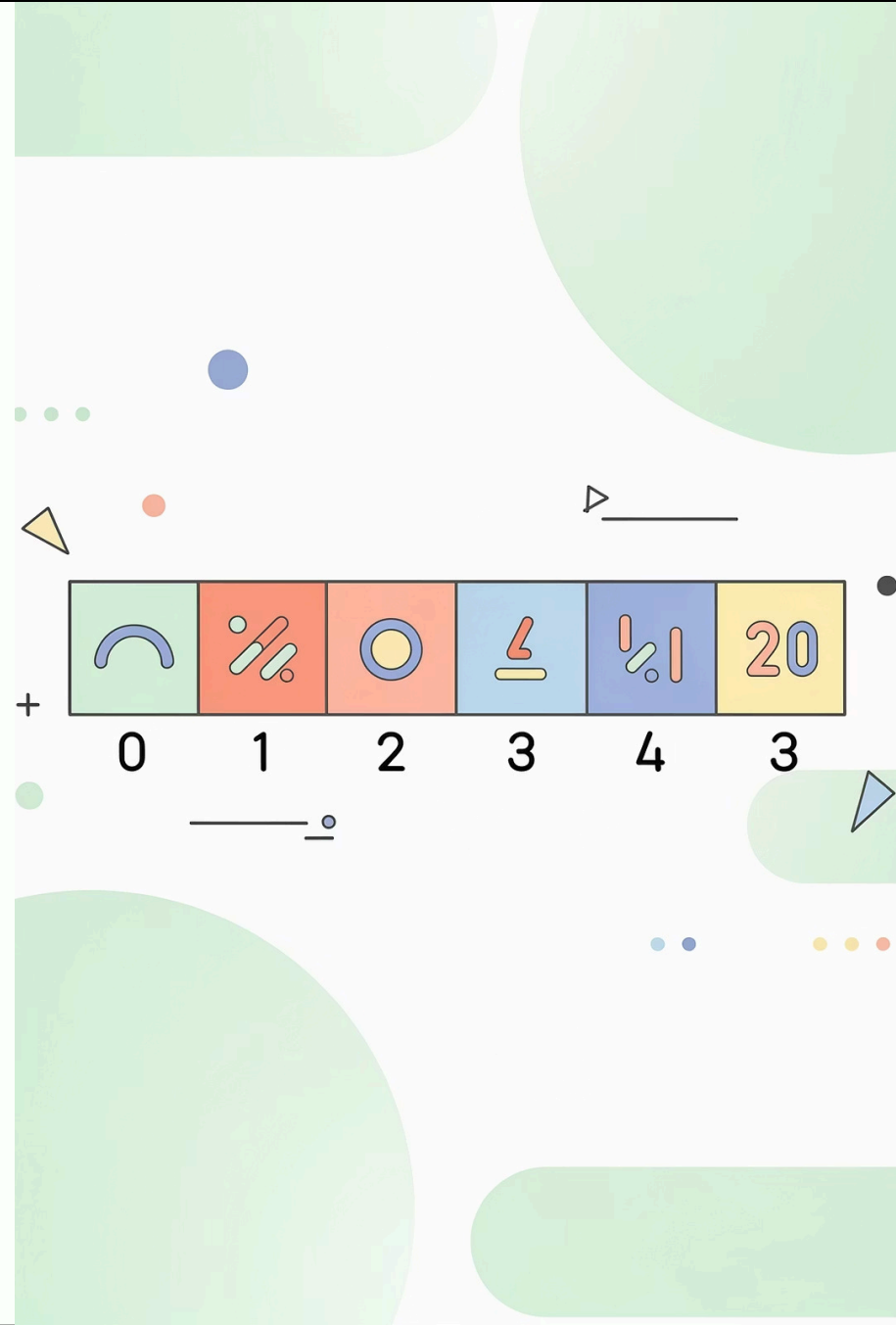
"Скільки програмістів
треба, щоб замінити
лампочку? Жодного — це
апаратна проблема.",

"Bug — це не помилка,
це недокументована
фіча."

]·

Масив — це список значень

- Зберігає кілька варіантів в одній змінній
- Кожен елемент має індекс: `jokes[0]`, `jokes[1]`...
- Зручно для відповідей, меню, команд



Випадкова відповідь через **Math.random**

```
bot.command("joke", (ctx) => {  
  const randomIndex = Math.floor(  
    Math.random() * jokes.length  
  );  
  ctx.reply(jokes[randomIndex]);  
});
```

01

Math.random()

Генерує число від 0 до 1

02

× jokes.length

Масштабує до розміру масиву

03

Math.floor()

Округлює до цілого індексу

04

jokes[randomIndex]

Повертає випадковий жарт

Цикли

```
const commands = ["start", "help", "joke"];

for (let i = 0; i < commands.length; i++) {
  console.log(commands[i]);
}
```

```
// Виведе:
// start
// help
// joke
```

Де цикл знадобиться у боті?

- Автоматична генерація пунктів меню
- Перебір списку команд
- Обробка масиву повідомлень

📌 `i` – це лічильник. Починає з 0, бо масиви в JS теж з 0.

```
project["node.jss.jog" {
{
  nppm insstall[
    "rofeect: dovel{305"}do.[];
    ioodrt_ic project_ssc(2020516°));
    isdeel_6d_1829998);
    ioddrs_dc contjejed oing I(Es";
    isdeet.8c_1834846));
    isdecl_6SC_52128S"))
  };
  "/s/conf{::cennn"; parrpakog_ision"" {
    sspet.iss({});
    /fnponc: /odre(2608pBZ"))[]";
    ""Teore:(nodje.jsn")56"), "ndex_js" import"js,server"}
  }
}
```

node-project

src

index_js

routes

controllers

config

package_json

server_js

README_md

Модулі в Node.js

messages.js

```
const jokes = [
  "Жарт 1",
  "Жарт 2"
];

module.exports = jokes;
```

bot.js

```
const jokes =
require("../messages");
// jokes вже доступні тут!
```

Навіщо ділити код на файли?

Чистота

Кожен файл – своя зона відповідальності

Масштаб

Великий бот = багато файлів

Повторне використання

Один модуль – для всього проєкту

Що таке модуль?

Модуль — це окремий файл із кодом

У Node.js кожен файл .js автоматично є модулем. Модулі допомагають розділяти програму на частини, щоб код був зрозумілішим і зручнішим у підтримці.

Без модулів

Весь код в одному файлі – важко читати, важко підтримувати

З модулями

Кожна частина програми – в окремому файлі зі своєю відповідальністю

Типова структура проєкту бота

```
my-bot/  
  bot.js      - головний файл  
  messages.js - тексти та масиви  
  commands.js - обробники команд  
  config.js   - налаштування
```

❏ **Кожен .js файл у Node.js — це вже модуль. Нічого додатково встановлювати не потрібно.**

Експорт коду з модуля

Файл `messages.js`

```
const jokes = [  
  "Жарт 1",  
  "Жарт 2",  
  "Жарт 3"  
];  
  
module.exports = jokes;
```

Ми робимо масив `jokes` доступним для інших файлів через `module.exports`.

Як працює `module.exports`?

01

Створюємо дані

Оголошуємо змінну, масив або функцію у файлі

02

Експортуємо

`module.exports` = назва – робимо доступним ззовні

03

Готово до використання

Інші файли можуть підключити цей модуль



Можна експортувати будь-що: масив, об'єкт, функцію або навіть рядок

Імпорт модуля через **require**

Файл **bot.js**

```
const jokes = require("./messages");  
  
console.log(jokes[0]); // "Жарт 1"
```

"./messages" означає файл messages.js у цій же папці. Крапка і слеш – це відносний шлях.

Що означає **"./messages"**?



• (крапка)

Поточна папка, де знаходиться файл



/ (слеш)

Розділювач шляху до файлу



messages

Назва файлу без розширення .js



Для вбудованих модулів Node.js (наприклад fs, path) крапку не пишуть: `require("fs")`

Модулі у Telegram-боті — приклад

messages.js

```
const jokes = ["Жарт 1", "Жарт 2"];

module.exports = jokes;
```

bot.js

```
const jokes = require("./messages");

bot.command("joke", (ctx) => {
  ctx.reply(jokes[0]);
});
```

Навіщо ділити код на модулі?

- Розділяти великий код на файли – легше читати
- Перевикористовувати функції в різних місцях
- Тримати логіку бота структурованою
- Легше знаходити і виправляти помилки

Типовий розподіл у боті:

1

bot.js

2

commands.js

3

messages.js

4

config.js



Чим більший бот — тим важливіше правильно розділяти код на модулі

Обробка кількох команд

```
bot.command("start", (ctx) => {  
  ctx.reply("Я працюю!");  
});  
  
bot.command("help", (ctx) => {  
  ctx.reply(  
    "Доступні команди:\n/start\n/help\n/joke"  
  );  
});  
  
bot.command("joke", (ctx) => {  
  const i = Math.floor(Math.random() * jokes.length);  
  ctx.reply(jokes[i]);  
});
```

Масштабування бота

- Кожна команда – окремий обробник
- Додати нову команду – додати один блок коду
- Структура залишається чистою

📌 Чим більше команд – тим важливіше ділити код на модулі.

МІНІ-ПРОЄКТ 1

Бот із трьома командами

1

Три команди

Реалізує `/start`, `/help`, `/joke`

2

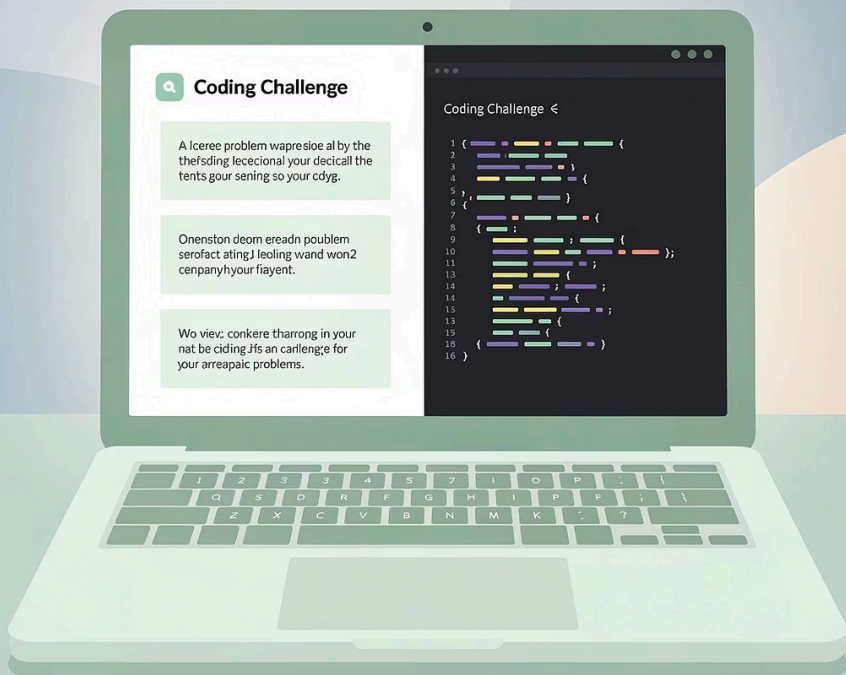
Масив відповідей

Жарти або факти –
зберігати у масиві

3

Запусти і протестуй

Надішли кожну команду у Telegram



Бот із випадковими відповідями

Завдання

```
// Масив із 5+ відповідей
const answers = [ ... ];

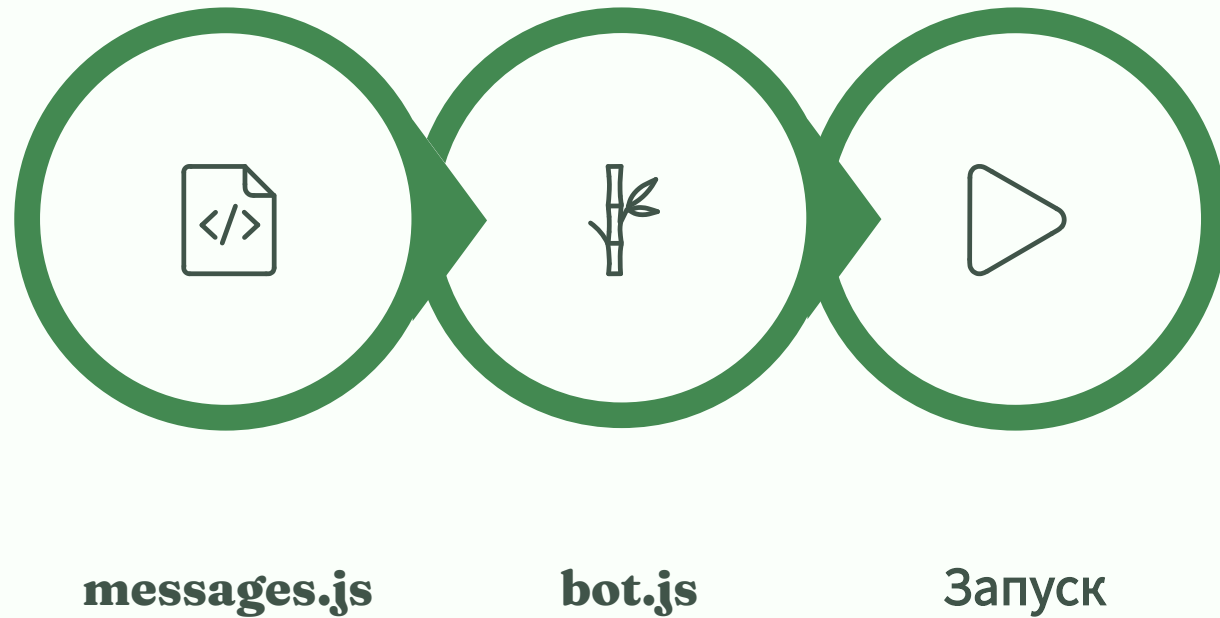
// При команді /random —
// видати випадкову відповідь
bot.command("random", (ctx) => {
  // твій код тут
});
```

Що відпрацьовуємо

- `Math.random() + Math.floor()`
- Робота з індексами масиву
- Обробка команди через `bot.command`

📌 **Бонус:** зроби масив із 10+ різних відповідей для більшої різноманітності.

Код у модулях



Розділи бота на два файли: `messages.js` зберігає дані, `bot.js` містить логіку. Переконайся, що все працює після рефакторингу.

Дебаг і типові помилки

✗ Неправильний токен

Перевір токен від @BotFather – жодного зайвого символу

✗ `bot.launch()` не викликаний

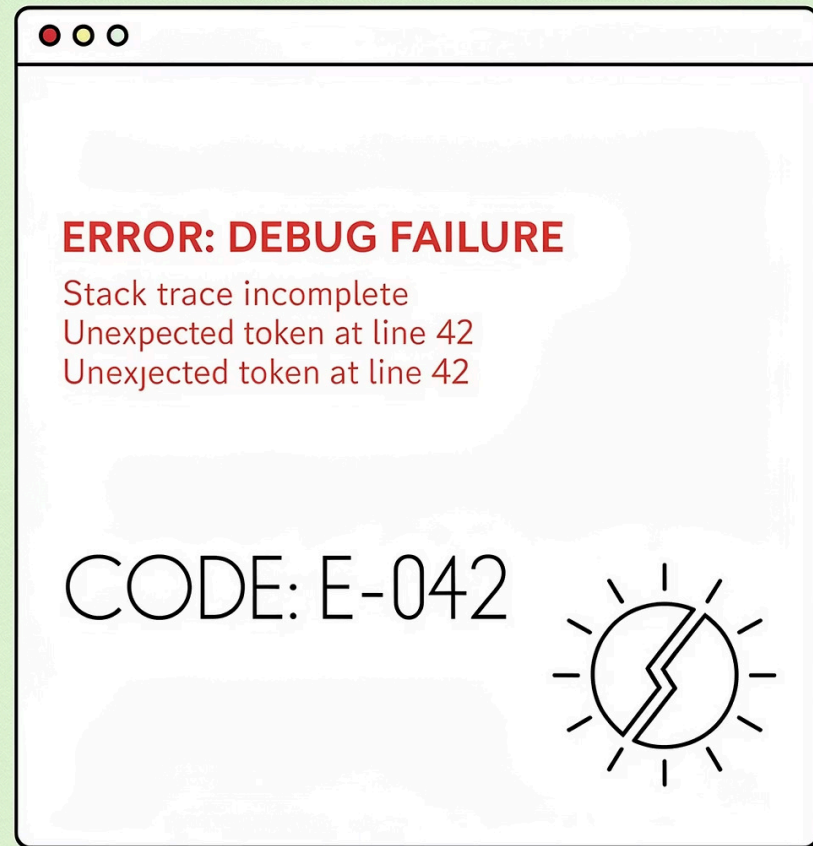
Без цього рядка бот ніколи не запуститься

✗ Синтаксична помилка

Пропущена дужка або лапка – термінал покаже рядок

✗ Бот не запущений

Запусти `node bot.js` у терміналі перед тестуванням



Підсумок модуля



Змінні

`const` і `let` у реальному боті



Умови

`if/else` для логіки відповідей



Функції

Чистий і структурований код



Масиви

Кілька відповідей в одній змінній



Цикли

Перебір команд і даних



Модулі

`require` і `module.exports`



Ти вже пишеш справжніх Telegram-ботів на Node.js. 🚀

Кнопки, повідомлення, сесії та логіка Telegram-ботів

У цьому модулі бот стане справжньо інтерактивним: з кнопками, медіа, іграми та збереженням стану користувача.



Кнопки

Reply та Inline клавіатури



Медіа

Фото, відео, файли



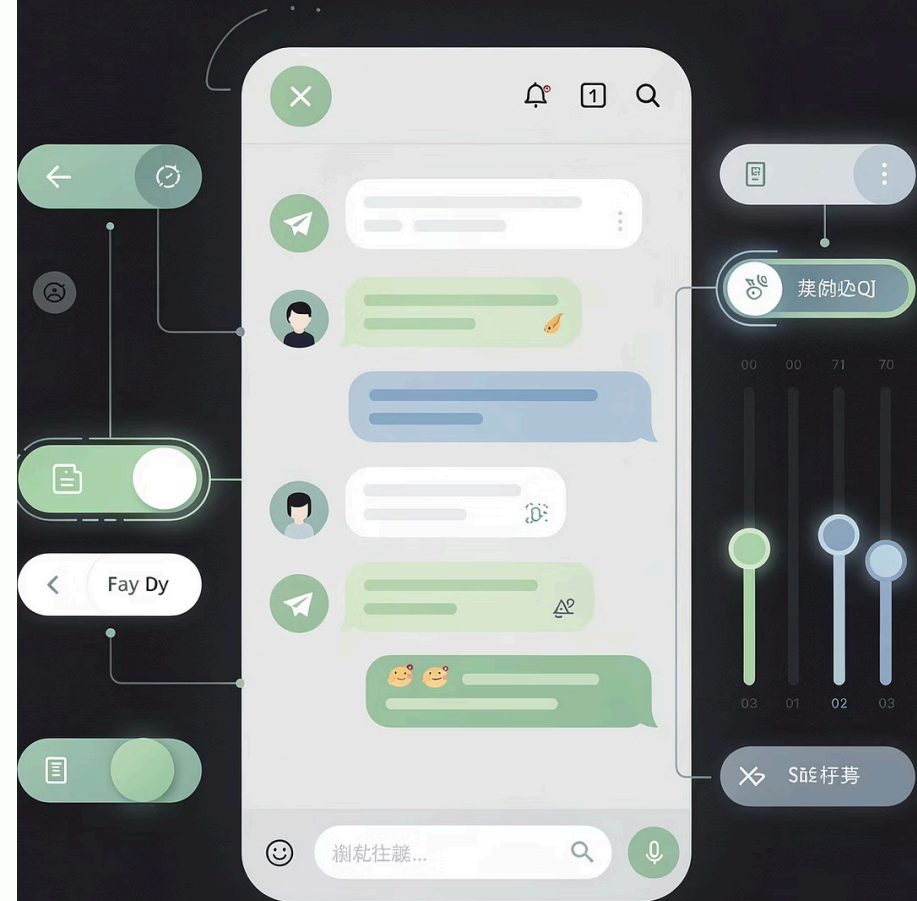
Сесії

Збереження стану користувача



Ігри

Вікторини та міні-ігри



Reply клавiатура (звичайні кнопки)

```
bot.start((ctx) => {  
  ctx.reply("Обери дію", {  
    reply_markup: {  
      keyboard: [  
        ["Інфо", "Допомога"],  
        ["Жарт"]  
      ],  
      resize_keyboard: true  
    }  
  });  
});
```

Що таке Reply клавiатура?

- Замінює стандартну клавiатуру користувача
- Кнопки відображаються внизу екрана
- Натискання надсилає текст кнопки як повідомлення
- `resize_keyboard: true` – підлаштовує розмір під кнопки

📌 **Reply клавiатура — найпростіший спосіб додати кнопки до бота**

Inline КНОПКИ

```
const { Markup } = require("telegraf");

bot.start((ctx) => {
  ctx.reply(
    "Меню",
    Markup.inlineKeyboard([
      Markup.button.callback("Жарт", "joke"),
      Markup.button.callback("Факт", "fact")
    ])
  );
});
```

Reply vs Inline — різниця

Reply клавіатура

Замінює клавіатуру телефону. Надсилає текст як повідомлення.

Inline КНОПКИ

З'являються прямо під повідомленням. Надсилають callback_data, не текст.

Обробка натискання кнопки

```
bot.action("joke", (ctx) => {  
  ctx.reply("Ось випадковий жарт");  
});
```

Що таке **callback_data**?

- `bot.action()` слухає натискання inline кнопок
- "joke" – це `callback_data`, яку ми задали у `Markup.button.callback`
- `ctx.reply()` надсилає відповідь користувачу
- Можна обробляти скільки завгодно різних кнопок

📄 `callback_data` — це **"секретний код" кнопки, який бот отримує при натисканні**

ПРАКТИКА

Меню навігації

Завдання: створити меню з кнопками



Nacheure



Stakenurse



Choring Beat



Utestany



/start

Бот вітає користувача і показує меню



Інфо

Бот надсилає інформацію про себе



Допомога

Бот показує список доступних команд



Кожна кнопка повинна викликати окрему відповідь бота через `bot.action()`

Надсилання картинок

```
bot.command("photo", (ctx) => {  
  ctx.replyWithPhoto("https://picsum.photos/300");  
});
```

Як працює **replyWithPhoto**?

- `ctx.replyWithPhoto()` надсилає зображення у чат
- Можна передати URL або `file_id` з Telegram
- Можна додати підпис через параметр `caption`
- Підтримує JPEG, PNG, GIF та інші формати

📄 `picsum.photos` – зручний сервіс для тестових зображень

Надсилання відео

```
bot.command("video", (ctx) => {  
  ctx.replyWithVideo(  
    "https://sample-  
videos.com/video123/mp4/240/big_buck_bunny_240p  
_1mb.mp4"  
  );  
});
```

Що потрібно знати про відео?

- `ctx.replyWithVideo()` надсилає відеофайл у чат
- Telegram підтримує MP4 формат
- Можна передати URL або `file_id`
- Максимальний розмір файлу – 50 МБ через URL
- Можна додати підпис через `caption`

❏ Для великих відео краще використовувати `file_id` після першого завантаження

Надсилання файлів

```
bot.command("file", (ctx) => {  
  ctx.replyWithDocument(  
    "https://example.com/file.pdf"  
  );  
});
```

Боти можуть надсилати будь-які файли

- `ctx.replyWithDocument()` надсилає файл будь-якого типу
- PDF, DOCX, ZIP, MP3 – все підтримується
- Можна передати URL або `file_id`
- Файл відображається як вкладення у чаті



PDF



ZIP



MP3



XLSX

Читання повідомлень користувача

```
bot.on("text", (ctx) => {  
  const text = ctx.message.text;  
  ctx.reply("Ти написав: " + text);  
});
```

Що таке **ctx.message.text**?

- `bot.on("text")` спрацьовує на будь-яке текстове повідомлення
- `ctx.message.text` – це текст, який написав користувач
- `ctx.from.first_name` – ім'я користувача
- `ctx.from.id` – унікальний ID користувача
- `ctx.chat.id` – ID чату



ctx – це контекст повідомлення. Він містить всю інформацію про відправника і чат.

Бот-чат

Завдання: створити бота, який відповідає на різні слова

привіт

Бот відповідає: "І тобі привіт!"



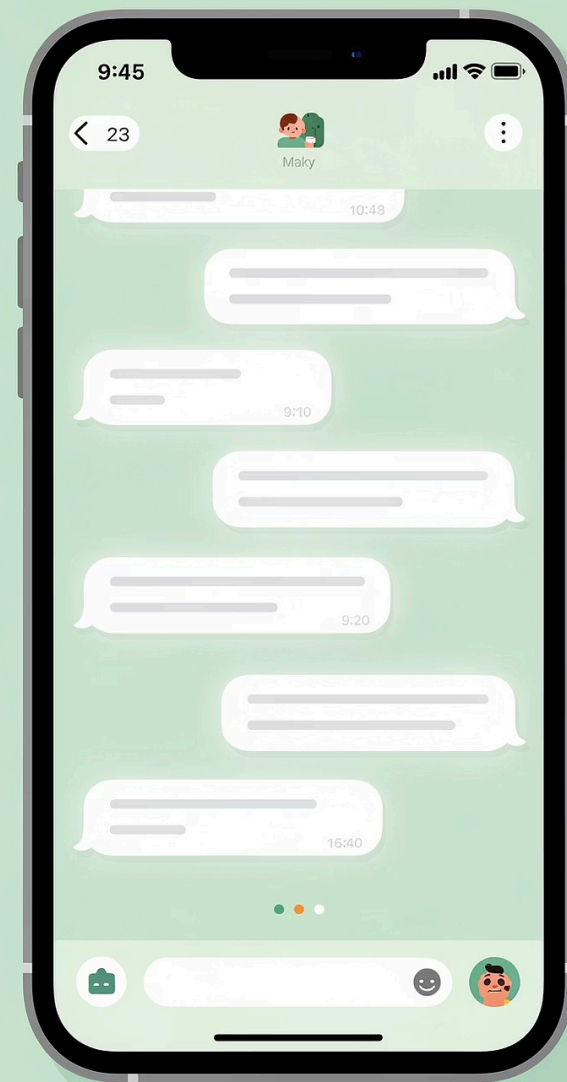
жарт

Бот надсилає випадковий жарт з масиву

help

Бот показує список доступних команд

📌 Використати if/else або switch для обробки різних слів



Навіщо потрібні сесії?

Проблема

Бот не пам'ятає попередніх повідомлень.
Кожне повідомлення – це окремий запит.

Без сесій

Бот не знає, що ти вже відповів на перше питання анкети

З сесіями

Бот пам'ятає твої крок, ім'я, вік та інші дані

Де потрібні сесії?

-  Анкети та опитування
-  Ігри (рахунок, поточний рівень)
-  Кошик у магазині-боті
-  Авторизація та профіль користувача



Сесія – це об'єкт, де ми зберігаємо дані конкретного користувача під час роботи бота

Проста сесія на об'єкті

```
const sessions = {};  
  
bot.on("text", (ctx) => {  
  const userId = ctx.from.id;  
  
  if (!sessions[userId]) {  
    sessions[userId] = { step: 1 };  
  }  
  
  ctx.reply("Твій крок: " + sessions[userId].step);  
});
```

Як це працює?

01

Отримуємо **userId**

ctx.from.id – унікальний ID кожного користувача

02

Перевіряємо сесію

Якщо сесії немає – створюємо нову

03

Зберігаємо стан

sessions[userId].step зберігає поточний крок

04

Відповідаємо

Бот знає, на якому кроці знаходиться користувач

ПРАКТИКА

Бот-анкета

Логіка: бот задає питання по черзі

1

Крок 1

Бот запитує: "Як тебе звати?"

2

Крок 2

Бот запитує: "Скільки тобі років?"

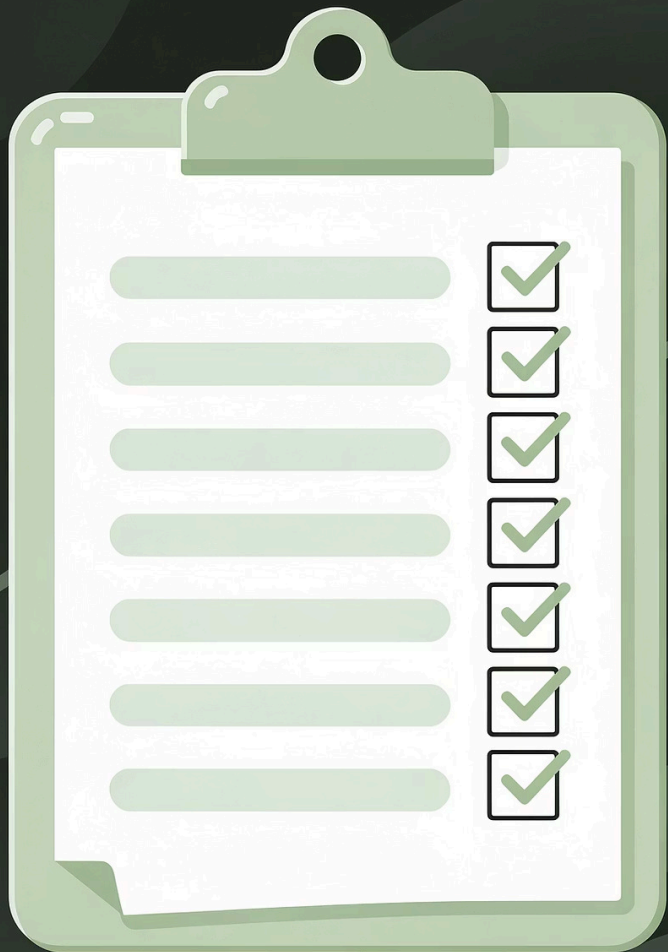
3

Крок 3

Бот запитує: "З якого ти міста?"



Після всіх відповідей бот показує: "Ім'я: ..., Вік: ..., Місто: ..."



Бот-анкета: повний код

bot.js

```
const { Telegraf } = require("telegraf");
const bot = new Telegraf("ТВИЙ_ТОКЕН");

const sessions = {};

bot.start((ctx) => {
  const id = ctx.from.id;
  sessions[id] = { step: 1 };
  ctx.reply("Як тебе звати?");
});

bot.on("text", (ctx) => {
  const id = ctx.from.id;
  const text = ctx.message.text;

  if (!sessions[id]) return;

  if (sessions[id].step === 1) {
    sessions[id].name = text;
    sessions[id].step = 2;
    ctx.reply("Скільки тобі років?");
  } else if (sessions[id].step === 2) {
    sessions[id].age = text;
    sessions[id].step = 3;
    ctx.reply("З якого ти міста?");
  } else if (sessions[id].step === 3) {
    sessions[id].city = text;
    const s = sessions[id];
    ctx.reply(
      "Ім'я: " + s.name +
      "\nВік: " + s.age +
      "\nМісто: " + s.city
    );
    delete sessions[id];
  }
});

bot.launch();
```

Як це працює?

01

bot.start()

Ініціалізує сесію і запитує ім'я

02

step === 1

Зберігає ім'я, переходить до кроку 2

03

step === 2

Зберігає вік, переходить до кроку 3

04

step === 3

Зберігає місто і виводить підсумок

Бот-довідник

Завдання: зробити бота з меню та кнопками



/start

Показує привітання та inline-меню з кнопками



/help

Список всіх доступних команд бота



/info

Інформація про бота та його можливості



Користувач обирає розділ через кнопки, бот відповідає відповідним контентом



Random у JavaScript

```
const randomNumber = Math.floor(  
  Math.random() * 10  
) + 1;  
  
console.log(randomNumber); // від 1 до 10
```

Як це працює?

01

Math.random()

Генерує випадкове число від 0 до 1 (наприклад, 0.742)

02

*** 10**

Множимо на 10, щоб отримати число від 0 до 9.99

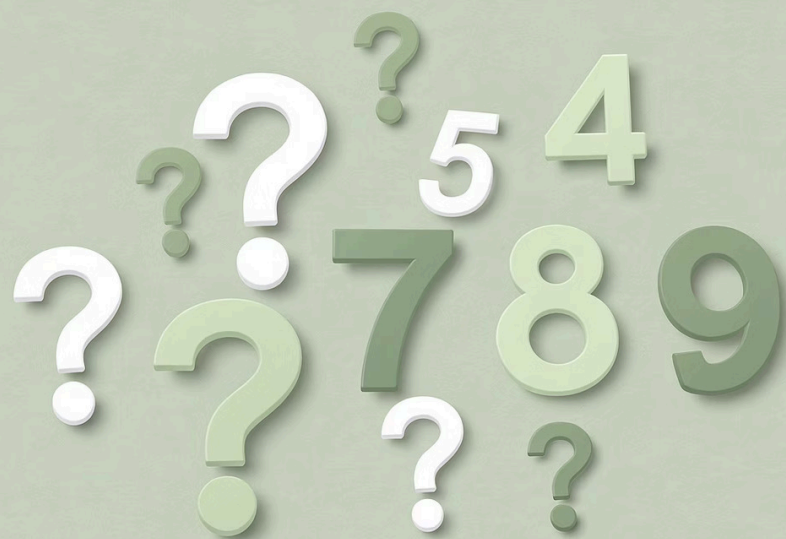
03

Math.floor() + 1

Округлюємо вниз і додаємо 1 → отримуємо від 1 до 10



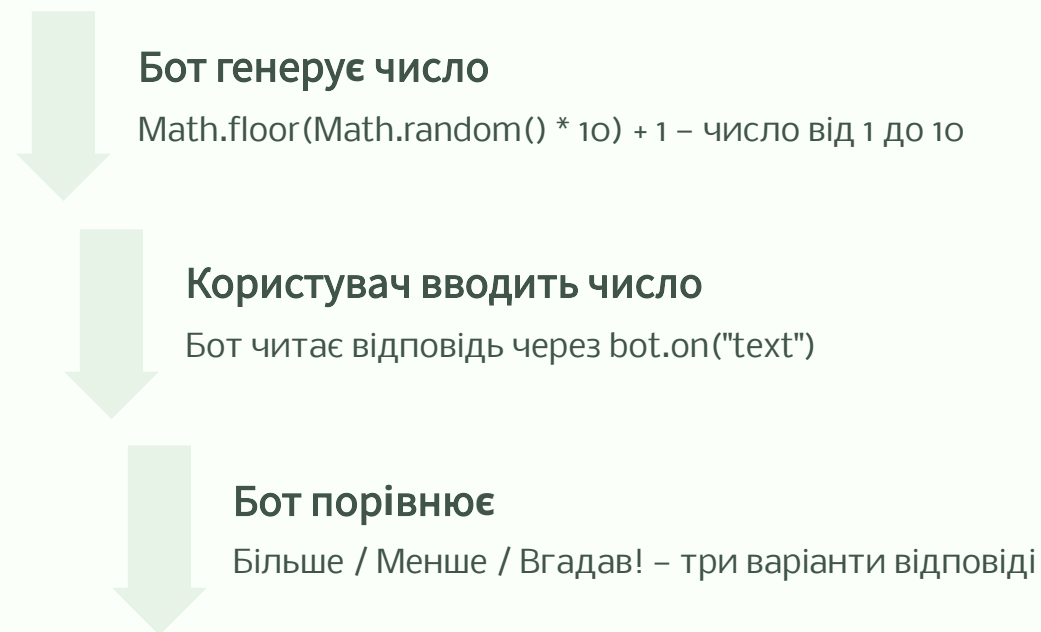
Math.random() * array.length – класичний спосіб отримати випадковий елемент масиву



ПРАКТИКА

Гра "Вгадай число"

Логіка гри:



- ❑ Зберігати загадане число у сесії: `sessions[userId].secret = randomNumber`

Кнопки-варіанти (quiz style)

```
bot.command("quiz", (ctx) => {  
  ctx.reply(  
    "Скільки буде 2 + 2?",  
    Markup.inlineKeyboard([  
      Markup.button.callback("3", "q1"),  
      Markup.button.callback("4", "q2"),  
      Markup.button.callback("5", "q3")  
    ])  
  );  
});
```

Як побудувати вікторину?

- Питання надсилається як текст повідомлення
- Варіанти відповідей – inline кнопки
- Кожна кнопка має свій callback_data (q1, q2, q3)
- bot.action() обробляє натискання кожної кнопки

❏ Можна додати кілька питань, зберігаючи рахунок у сесії

Обробка відповіді у вікторині

```
bot.action("q2", (ctx) => {  
  ctx.reply("Правильно!");  
});  
  
bot.action("q1", (ctx) => {  
  ctx.reply("Неправильно");  
});
```

Що відбувається при натисканні?

Правильна відповідь (q2)

Бот надсилає повідомлення про успіх і може перейти до наступного питання

Неправильна відповідь (q1, q3)

Бот повідомляє про помилку і може дати підказку або повторити питання

- ❏ Можна додати лічильник правильних відповідей у сесії користувача

Ідеї для розширення бота



Бот-вікторина

Питання з варіантами відповідей та підрахунком балів



Бот-гра

Вгадай число, хрестики-нулики, текстові пригоди



Бот-довідник

Структуроване меню з корисною інформацією



Бот-меми

Надсилає випадкові меми або картинки за командою



Бот-рандомні факти

Цікаві факти з масиву або зовнішнього API



Підсумок модуля

Після цього модуля ти вмієш будувати справжньо інтерактивних Telegram-ботів!



Кнопки

Reply та Inline клавіатури



Читання повідомлень

`bot.on("text")` та `ctx.message.text`



Медіа

Фото, відео та файли



Інтерактивні меню

Навігація через inline кнопки



Random

`Math.random()` для ігор та жартів



Стан користувача

Сесії для збереження даних



Міні-ігри

Вікторини та гра "Вгадай число"



Ти готовий створювати повноцінних інтерактивних Telegram-ботів! 🚀

Підключення API

Що таке API?

API (Application Programming Interface) дозволяє отримувати дані з інтернету – погоду, картинки, цитати тощо.

Бот може використовувати ці дані і надсилати їх користувачу.



Погода

Отримуй актуальну температуру та прогноз



Картинки

Надсилає фото котиків, мемів та іншого



Цитати

Випадкові мотиваційні або смішні цитати

axios для запитів

Встановлення

```
npm install axios
```

Підключення

```
const axios = require("axios");
```

Навіщо axios?

01

Надсилає **HTTP**-запит до **API**

02

Отримує відповідь у форматі **JSON**

03

Ти використовуєш дані у боті

Приклад API (котики)

```
const axios = require("axios");

axios.get("https://api.thecatapi.com/v1/images/search")
  .then((response) => {
    console.log(response.data);
  });
```

Як це працює?

01

axios.get()

Надсилає GET-запит до API котиків

02

response.data

Містить масив із даними про зображення

03

console.log()

Виводить URL картинки у термінал

Надсилання котика в боті

```
const axios = require("axios");

bot.command("cat", async (ctx) => {
  const response = await axios.get(
    "https://api.thecatapi.com/v1/images/search"
  );
  const imageUrl = response.data[0].url;
  ctx.replyWithPhoto(imageUrl);
});
```

Ключові моменти

- `async/await` – сучасний спосіб роботи з асинхронним кодом
- `response.data[0].url` – перший елемент масиву, поле `url`
- `ctx.replyWithPhoto()` – надсилає зображення у чат
- Кожен виклик `/cat` – нова випадкова картинка

API погоди

```
bot.command("weather", async (ctx) => {  
  const response = await axios.get(  
    "https://api.open-meteo.com/v1/forecast" +  
    "?latitude=50&longitude=30" +  
    "&current_weather=true"  
  );  
  const temp =  
    response.data.current_weather.temperature;  
  ctx.reply("Температура: " + temp + "°C");  
});
```

Що тут відбувається?

01

URL з параметрами

latitude і longitude – координати Києва

02

current_weather

Поточна погода у відповіді API

03

temperature

Числове значення температури у °C

Практика: API у боті

Завдання

Зроби команду /cat або /weather, яка отримує дані з API і надсилає користувачу.

```
// Варіант 1: котик
bot.command("cat", async (ctx) => {
  // отримай URL з thecatapi.com
  // надішли фото через replyWithPhoto
});

// Варіант 2: погода
bot.command("weather", async (ctx) => {
  // отримай температуру з open-meteo.com
  // надішли текст через ctx.reply
});
```

Підказки

- Не забудь `npm install axios` перед початком
- Використовуй `async/await` для запитів
- Перевір `response.data` у `console.log` спочатку

Збереження даних

Навіщо зберігати дані?

Бот може зберігати інформацію, щоб пам'ятати користувачів або результати між сесіями.



Користувачі

Пам'ятати ім'я, налаштування, прогрес



Статистика

Рахувати запуски, повідомлення, дії



Результати

Зберігати рахунок у грі або відповіді



JSON файл як база

Файл `data.json`

```
{  
  "visits": 0,  
  "users": {}  
}
```

Це простий спосіб зберігати дані між запусками бота. JSON – текстовий формат, зрозумілий і людині, і JavaScript.

Переваги JSON-файлу



Простота

Не потрібна база даних – лише файл



Читабельність

Можна відкрити і переглянути вручну



Швидкий старт

Ідеально для навчальних проєктів

Читання JSON

```
const fs = require("fs");

const data = JSON.parse(
  fs.readFileSync("data.json")
);

console.log(data.visits); // 0
console.log(data.users); // {}
```

Що тут відбувається?

01

fs.readFileSync()

Читає вміст файлу як текстовий рядок

02

JSON.parse()

Перетворює рядок у JavaScript-об'єкт

03

data.visits

Тепер можна звертатись до полів об'єкта

Запис у JSON

```
const fs = require("fs");

// Змінюємо дані
data.visits = 5;

// Записуємо назад у файл
fs.writeFileSync(
  "data.json",
  JSON.stringify(data, null, 2)
);
```

Параметри **JSON.stringify()**

DATA

Об'єкт, який потрібно зберегти

NULL

Замінник (не використовується)

2

Відступ у 2 пробіли для читабельності

Практика: лічильник запусків

Завдання

```
bot.start((ctx) => {  
  data.visits++;  
  fs.writeFileSync(  
    "data.json",  
    JSON.stringify(data)  
  );  
  ctx.reply("Запусків: " + data.visits);  
});
```

Як це зробити?

01

Створи файл **data.json** з **{ "visits": 0 }**

02

Зчитай дані через **fs.readFileSync** + **JSON.parse**

03

У **bot.start** збільшуй **data.visits++**

04

Записуй оновлені дані через **fs.writeFileSync**

Практика: збереження повідомлень

Завдання

```
bot.on("text", (ctx) => {  
  data.users[ctx.from.id] =  
    ctx.message.text;  
  fs.writeFileSync(  
    "data.json",  
    JSON.stringify(data)  
  );  
  ctx.reply("Збережено");  
});
```

Що тут відбувається?

- `ctx.from.id` – унікальний ID користувача в Telegram
- `data.users[id] = text` – зберігаємо останнє повідомлення
- Кожен користувач має свій запис у об'єкті `users`
- Після перезапуску бота дані зберезуться у файлі

Міні-проєкт: інтерактивна гра

Ідея гри

Бот загадує випадкове число від 1 до 10. Користувач вводить своє число. Бот відповідає «більше» або «менше» поки не вгадає.

Логіка

- Генерує число через `Math.random()`
- Зчитує відповідь через `bot.on text`
- Порівнює через `if/else` і відповідає

Що потрібно реалізувати?



Загадати число

`Math.random()` при старті гри



Читати відповідь

`bot.on text + parseInt()`



Порівняти

`if/else` для більше, менше, вгадав



Міні-проєкт: бот-асистент

Команди бота



/cat

Надсилає випадкову картинку котика з API



/weather

Показує поточну температуру в Києві



/help

Виводить список доступних команд

Структура проєкту

- bot.js – головний файл бота
- data.json – збереження даних
- npm install telegraf axios
- Кожна команда – окремих bot.command()



Підсумок

Бот тепер вміє робити набагато більше – від запитів до API до збереження даних!



Робота з API

Отримувати дані з інтернету через axios



Надсилати дані

Фото, погода та інша інформація з мережі



Зберігати інформацію

JSON-файл як проста база даних



Проста логіка

if/else, цикли, функції у реальному боті



Міні-ігри

Вгадай число та інші інтерактивні механіки



Ти готовий створювати повноцінних Telegram-ботів з реальними даними! 🚀